

Estruturas de Dados: Introdução



Estruturas de Dados

Edson da Silva Castro

IFMS

Instituto Federal de Educação Ciência e Tecnologia de Mato Grosso do Sul

Três Lagoas, 2024

Lista

Definição

"Em ciência da computação, **uma lista** é uma **estrutura de dados linear** utilizada para armazenar e organizar dados em um computador. Uma estrutura do tipo lista é uma **sequência de elementos do mesmo tipo**." (grifo nosso) (BACKES, 2016)

Lista

Observações acerca das listas

- Podem possuir elementos repetidos;
- Podem ser ordenadas ou não (depende da aplicação);
- Seja n o número de elementos da lista: $n \geq 0$. Se $n=0$ dizemos que a lista está vazia.

nós da lista

Os elementos de uma lista também são chamados de **nós** (*nodes* em inglês).

Tipos de Listas

As listas diferem segundo três critérios:

- Ordem de inserção/remoção dos elementos;
- Alocação de memória;
- Tipo de acesso aos elementos;

Tipos de Listas

Quanto à ordem de inserção/remoção dos elementos

- **Convencional:** elementos inseridos/removidos em qualquer posição da lista;

Tipos de Listas

Quanto à ordem de inserção/remoção dos elementos

- **Convencional:** elementos inseridos/removidos em qualquer posição da lista;
- **Fila:** os elementos da lista são removidos na mesma ordem que eles foram inseridos, ou seja, **primeiro a entrar, primeiro a sair**;

Tipos de Listas

Quanto à ordem de inserção/remoção dos elementos

- **Convencional:** elementos inseridos/removidos em qualquer posição da lista;
- **Fila:** os elementos da lista são removidos na mesma ordem que eles foram inseridos, ou seja, **primeiro a entrar, primeiro a sair**;
- **Pilha:** os elementos da lista são removidos na ordem inversa que eles foram inseridos, ou seja, **último a entrar, primeiro a sair**.

Tipos de Listas

Quanto ao momento de Alocação de Memória

- **Alocação estática:** elementos da lista são alocados na memória em tempo de compilação;
- **Alocação dinâmica:** alocação dinâmica sob demanda dos elementos da lista, em tempo de execução;

Tipos de Listas

Quanto ao tipo de acesso aos elementos

- **Sequencial:** elementos inseridos de forma consecutiva na memória;

Tipos de Listas

Quanto ao tipo de acesso aos elementos

- **Sequencial:** elementos inseridos de forma consecutiva na memória;
- **Encadeada:** os elementos não necessariamente estão em áreas consecutivas da memória. Cada elemento necessita guardar um ponteiro para o próximo elemento da lista.

Principais operações sobre uma lista

As principais operações sobre uma lista são:

- Referente à lista:
 - criação;
 - destruição;
 - verificação do tamanho;
 - verificação se está vazia, ou cheia;
 - listagem dos elementos;
- Referente aos elementos da lista;
 - Inserção;
 - Remoção;
 - Busca;

Tipos de Listas

Os tipos mais comuns de implementação das listas são:

- Lista sequencial estática;
- Lista dinâmica encadeada;

Observação

As listas sequenciais estáticas são de fácil compreensão. Portanto, focaremos nossos esforços nas listas dinâmicas.

Para rever

Rever/Estudar

- Ponteiros
- Ponteiros para estruturas
- Ponteiros de ponteiros

Para refletir

Observe o seguinte código fonte. Num contexto de estruturas de dados reflita:

```
1  struct Estudante {  
2      int ID;  
3      float nota;  
4      char nome[60];  
5  };  
6  
7  struct No {  
8      struct Estudante estudante;  
9      struct No *proximo;  
10 } No;
```

Para refletir

- Para que serve tal estrutura de dados?
- Quais as possíveis operações?
- Como seriam implementadas as operações?
- Na hora de implementar, quais os detalhes importantes a observar?

Lista Dinâmica Encadeada

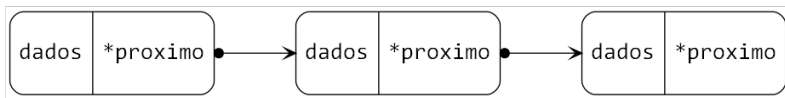
As listas dinamicamente encadeadas também são chamadas de listas ligadas (*linked lists* no inglês).

Principal característica das listas encadeadas

Cada elemento possui:

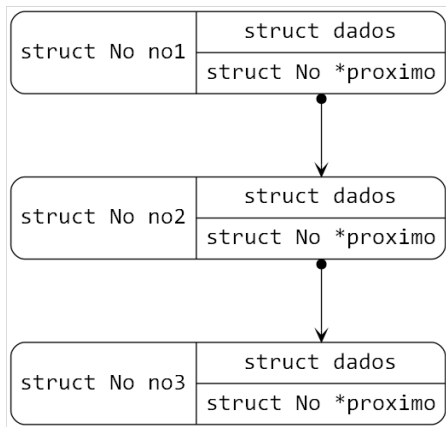
- uma estrutura que define os **dados** armazenados;
- e um **ponteiro para o próximo** elemento da lista.

Exemplo Simplificado de Lista Encadeada



Dados de uma Lista Dinâmica Encadeada

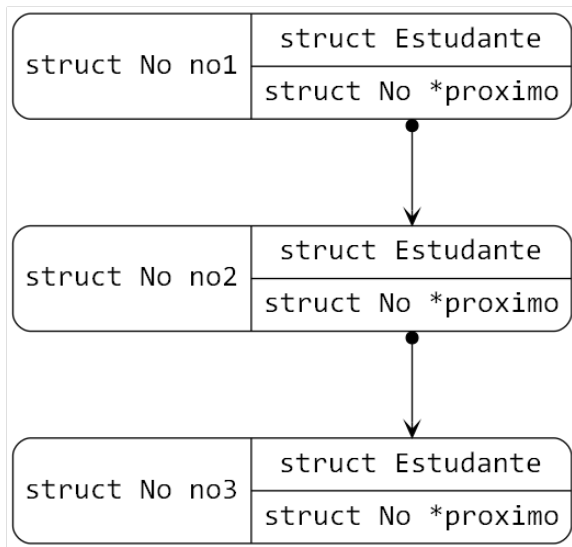
Os dados da lista podem ser estruturas complexas!

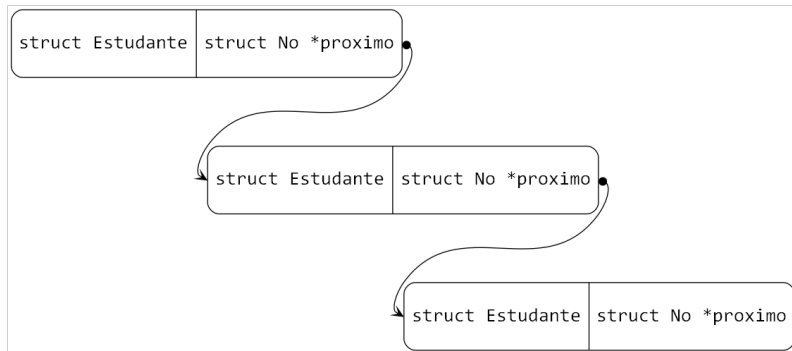


Declaração de uma Lista Encadeada com 3 nós

```
1 struct Estudante {
2     int ID;
3     float nota;
4     char nome[60];
5 };
6
7 struct No {
8     struct Estudante estudante;
9     struct No *proximo;
10 } No;
```

```
1 // Criação de 3 nós da lista encadeada
2 // (Ainda sem alocação dinâmica)
3 struct No no1, no2, no3;
4
5 no1.proximo = &no2;
6 no2.proximo = &no3;
7 no3.proximo = NULL;
```





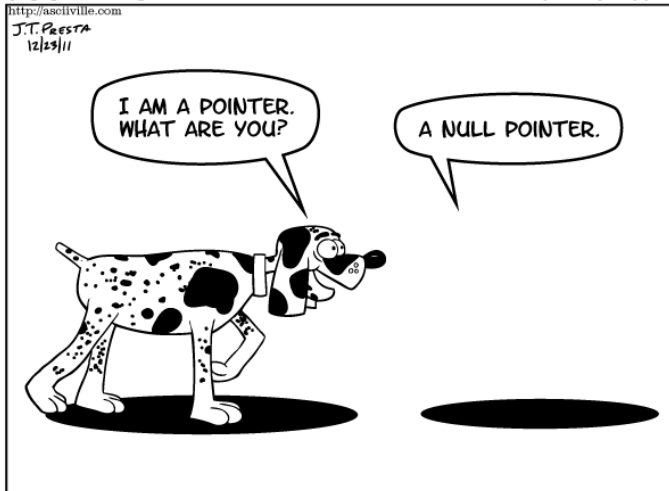
NULL Pointer

asciiville^{v2}

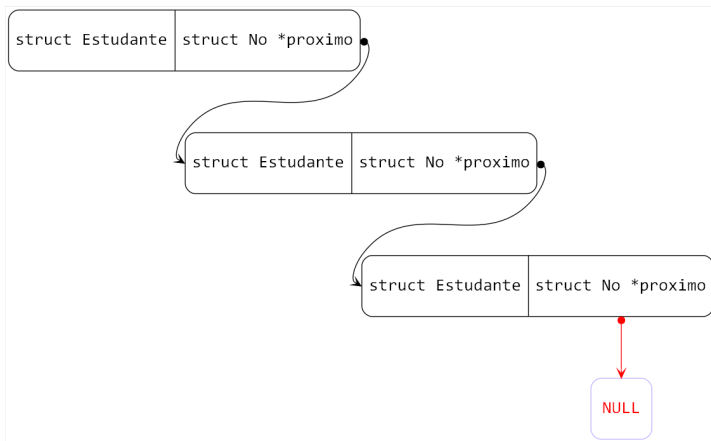
<http://asciiville.com>

J.T. PRESTA
12/23/11

“Null Pointer”



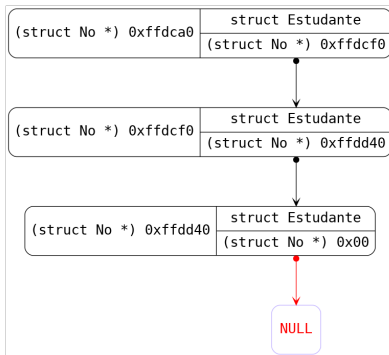
Último elemento da lista apontará para **NULL**

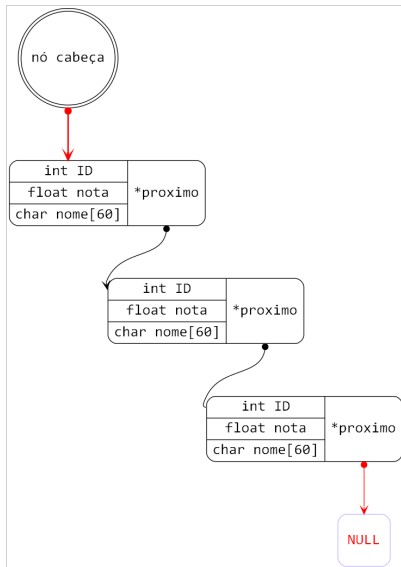


Hora de depurar para compreender melhor

Endereços de todos os nós:

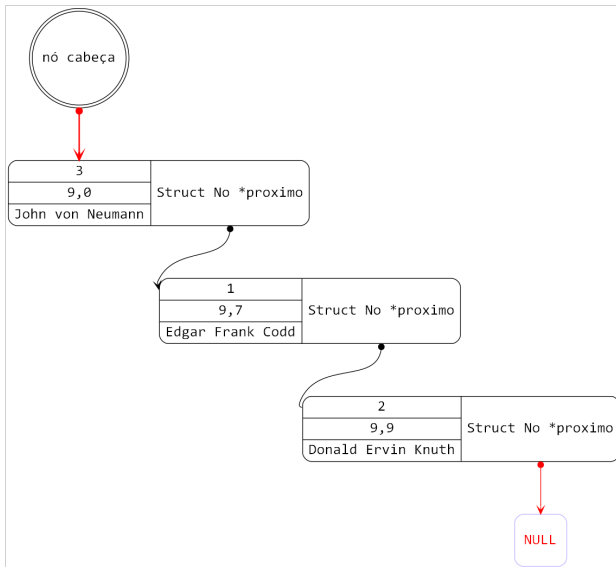
no1: 0x7fffffff dca0 próximo: 0x7fffffff dcf0
no2: 0x7fffffff dcf0 próximo: 0x7fffffff dd40
no3: 0x7fffffff dd40 próximo: (nil)





Inserindo os dados nos nós da lista

```
1 no1.estudante.ID = 3;
2 no1.estudante.nota = 9.0;
3 strcpy(no1.estudante.nome, "John von Neumann");
4
5 struct Estudante estudante2, estudante3;
6 estudante2.ID = 1;
7 estudante2.nota = 9.7;
8 strcpy(estudante2.nome, "Edgar Frank Codd");
9
10 estudante3.ID = 2;
11 estudante3.nota = 9.9;
12 strcpy(estudante3.nome, "Donald Ervin Knuth");
13
14 no2.estudante = estudante2;
15 no3.estudante = estudante3;
```



Um pouco mais de depuração para compreender ainda melhor

Sim, é possível depurar assim!

- `no1;`
- `no1 → proximo;`
- `no1 → proximo → proximo;`
- `*no1 → proximo;`
- `*no1 → próximo → proximo;`
- `(*no1 → próximo → proximo) → estudante.nome;`

Um pouco mais de depuração para compreender ainda melhor

```
(gdb) display no1
5: no1 = {estudante = {ID = 3, nota = 9,
    nome = "John von Neumann\000\000\000"}, proximo = 0x7fffffffddcf0}
(gdb) display no1->proximo
6: no1->proximo = (struct No *) 0x7fffffffddcf0
(gdb) display no1->proximo->proximo
7: no1->proximo->proximo = (struct No *) 0x7fffffffdd40
(gdb) display *no1->proximo
8: *no1->proximo = {estudante = {ID = 1, nota = 9.699999981,
    nome = "Edgar Frank Codd\000\000\000"}, proximo = 0x7fffffffdd40}
(gdb) display (*no1->proximo->proximo)->estudante.nome
9: (*no1->proximo->proximo)->estudante.nome = "Donald Ervin Knuth\000"
```

Cuidado para não acessar posições inválidas de memória!

Note que no nosso exemplo

`no1 → próximo → proximo → proximo == NULL`

Se `p == NULL`, `*p == ?`

Cuidado para não acessar posições inválidas de memória!

Note que no nosso exemplo

`no1 → próximo → proximo → proximo == NULL`

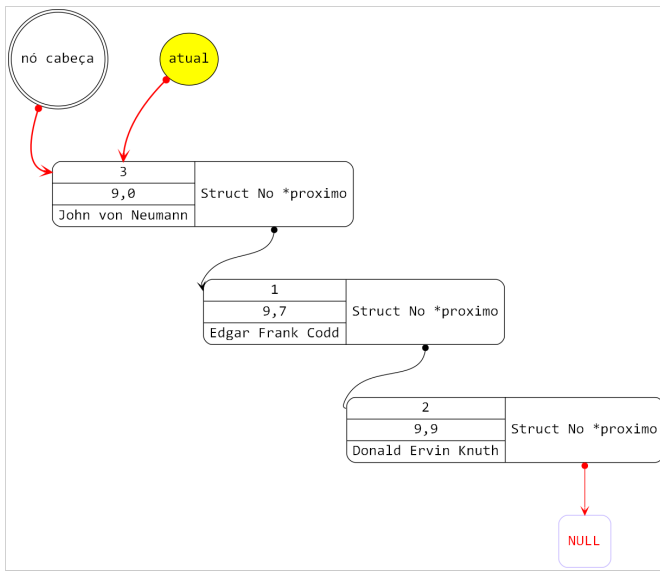
Se `p == NULL`, `*p == ?`

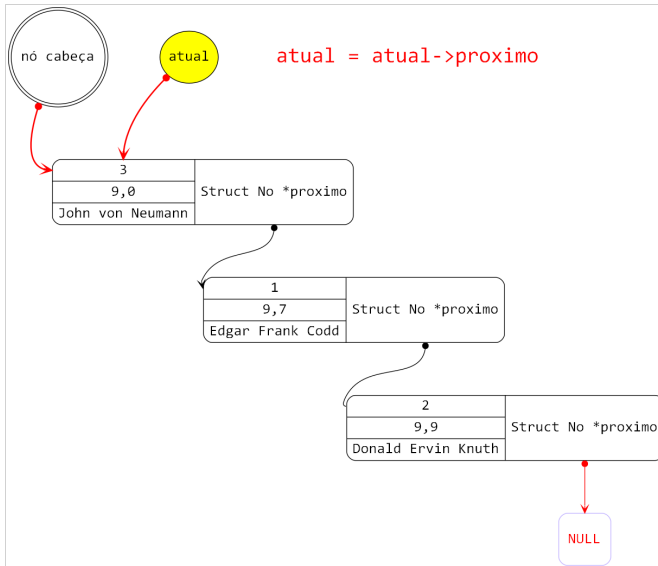


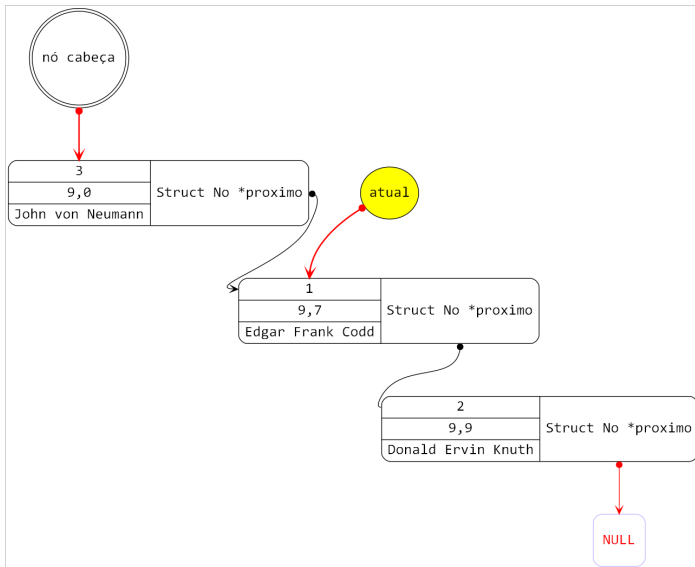
Cannot access memory at address

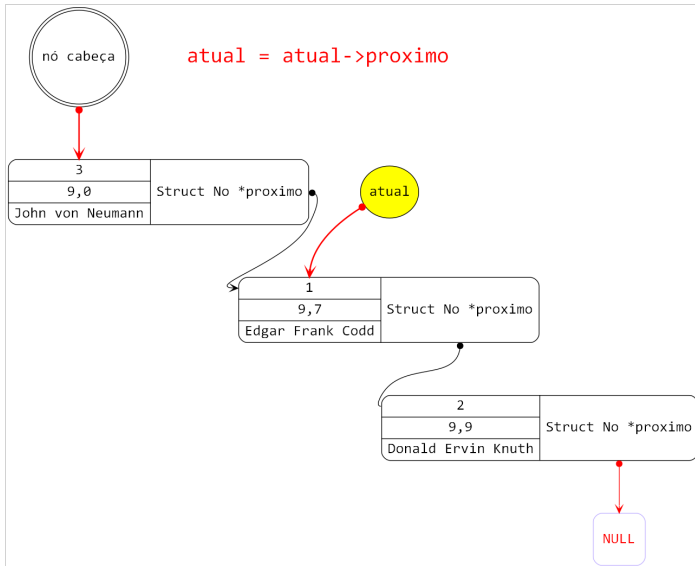
0x0 

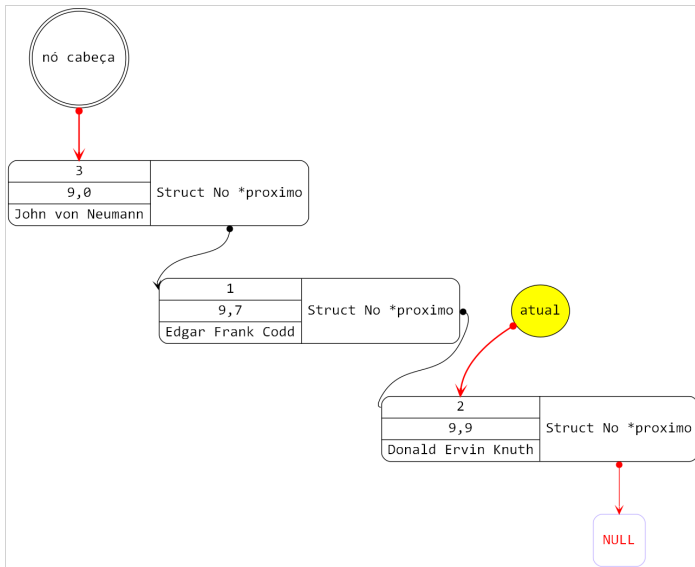
Percorrendo toda a Lista Dinâmica Encadeada

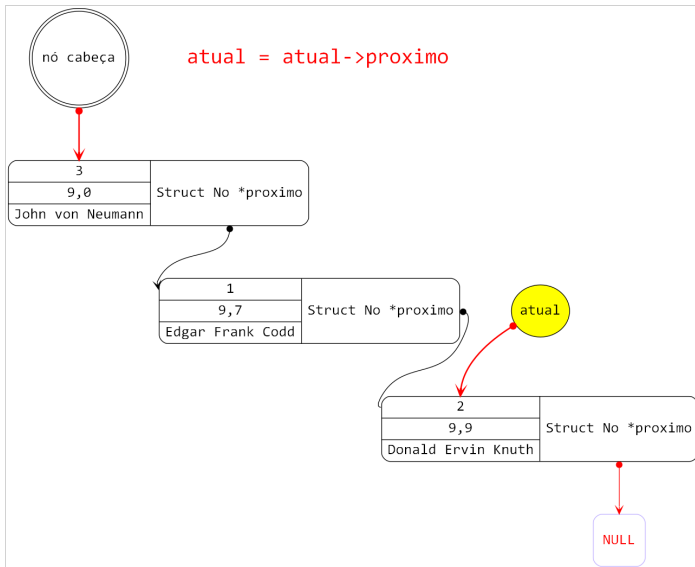


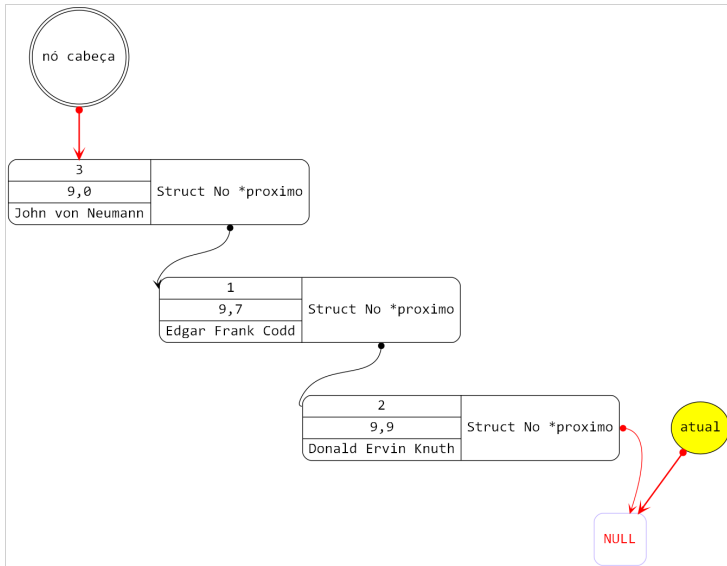


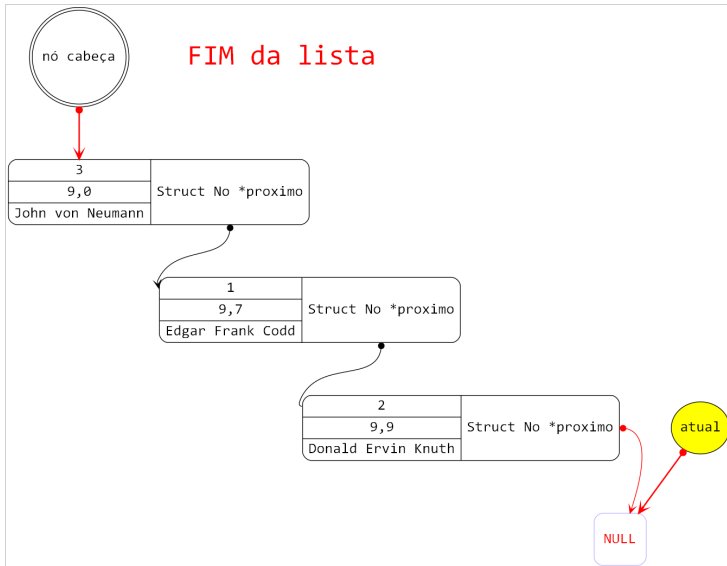












Impressão de todos os elementos da lista

```
1 struct No *atual;  
2 atual = cabeca;  
3  
4 printf("ID\tNOTA\tNOME\n");  
5 while (atual != NULL)  
6 {  
7     printf("%d\t", atual->estudante.ID);  
8     printf("%.2f\t", atual->estudante.nota);  
9     printf("%s\n", atual->estudante.nome);  
10  
11     atual = atual->proximo;  
12 }
```


Definição do tipo lista dinâmica encadeada

Antes de utilizar uma lista encadeada num programa é necessário **criar uma lista vazia**, através da **alocação de memória** para guardar o endereço do início da lista, que anteriormente chamamos de cabeça da lista.

Início da lista

Tal endereço de memória, que representa o início da lista, deverá ser um **ponteiro de ponteiro!**

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar

- Quando se trabalha com ponteiros, **desenhar** graficamente cada passo a implementar **facilita** "muuuuito".

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar

- Quando se trabalha com ponteiros, **desenhar** graficamente cada passo a implementar **facilita** *"muuuuito"*.
- Nas operações de **inserção e remoção** de elementos da lista **não é necessário movimentar** outros elementos de lugar, diferente do que ocorre nas listas estáticas!

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar

- Quando se trabalha com ponteiros, **desenhar** graficamente cada passo a implementar **facilita** "muuuuito".
- Nas operações de **inserção e remoção** de elementos da lista **não é necessário movimentar** outros elementos de lugar, diferente do que ocorre nas listas estáticas!
- Para inserir um elemento no final da lista, remover o elemento do fim, ou obter a quantidade de elementos da lista é necessário **percorrê-la até o final**; *Depois veremos estratégias para melhorar muito isto!*

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar

- Quando se trabalha com ponteiros, **desenhar** graficamente cada passo a implementar **facilita** "muuuuito".
- Nas operações de **inserção e remoção** de elementos da lista **não é necessário movimentar** outros elementos de lugar, diferente do que ocorre nas listas estáticas!
- Para inserir um elemento no final da lista, remover o elemento do fim, ou obter a quantidade de elementos da lista é necessário **percorrê-la até o final**; *Depois veremos estratégias para melhorar muito isto!*
- Uma lista só estará vazia se seu início apontar para NULL;

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar (continuação)

- Antes de manipular uma lista, devemos sempre verificar se ela é **válida** ($\neq \text{NULL}$), ou seja, se ocorreu sucesso na alocação de memória; Também é necessário proceder assim sempre que trabalhar com variáveis alocadas dinamicamente.

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar (continuação)

- Antes de manipular uma lista, devemos sempre verificar se ela é **válida** ($\neq \text{NULL}$), ou seja, se ocorreu sucesso na alocação de memória; Também é necessário proceder assim sempre que trabalhar com variáveis alocadas dinamicamente.
- Na **inserção ordenada** é necessário procurar o **local onde o dado será inserido**. Também é necessário **alterar alguns ponteiros**. Note que a inserção poderá ocorrer no início da lista em duas situações!

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar (continuação)

- Antes de manipular uma lista, devemos sempre verificar se ela é **válida** ($\neq \text{NULL}$), ou seja, se ocorreu sucesso na alocação de memória; Também é necessário proceder assim sempre que trabalhar com variáveis alocadas dinamicamente.
- Na **inserção ordenada** é necessário procurar o **local onde o dado será inserido**. Também é necessário **alterar alguns ponteiros**. Note que a inserção poderá ocorrer no início da lista em duas situações!
- A operação de **busca não remove** qualquer elemento da lista, apenas retorna o elemento procurado.

Implementação de listas dinamicamente encadeadas

Dicas e observações importantes na hora de implementar (continuação)

- Antes de manipular uma lista, devemos sempre verificar se ela é **válida** ($\neq \text{NULL}$), ou seja, se ocorreu sucesso na alocação de memória; Também é necessário proceder assim sempre que trabalhar com variáveis alocadas dinamicamente.
- Na **inserção ordenada** é necessário procurar o **local onde o dado será inserido**. Também é necessário **alterar alguns ponteiros**. Note que a inserção poderá ocorrer no início da lista em duas situações!
- A operação de **busca não remove** qualquer elemento da lista, apenas retorna o elemento procurado.
- Tanto a operação de busca quanto a de remoção devem retornar **NULL** caso os elementos procurados não forem encontrados.

Pergunta de um milhão de dólares

Válida para toda e qualquer atividade avaliativa!

O seguinte trecho de código foi retirado da função main(). O que significa cada um dos comandos das linhas 4, 5 e 6 se a condição da linha 2 for verdadeira?

```
1 float *nota = malloc( sizeof( float ) );  
2 if (plagio == 1)  
3 {  
4     free(nota);  
5     nota = NULL;  
6     return 0;  
7 }
```

Psiu, não pense palavrões proibidos para menores de 21 anos!

Referência Bibliográfica

BACKES, André. **Estrutura de dados descomplicada em linguagem C**. São Paulo: Addison Wesley, 2016.

Castro, Edson da Silva. **Uma intervenção baseada numa abordagem significativa do aprendizado de Algoritmos 1 no curso técnico de nível médio integrado em informática**. Especialização em Docência para a Educação Profissional e Tecnológica - Instituto Federal de Educação Ciência e Tecnologia de Mato Grosso do Sul, 2020.

RICARTE, Ivan Luiz Marques.

Estruturas de dados. 2008. Documento eletrônico. Disponível em: <https://wiki.dca.fee.unicamp.br/wiki/images/0/01/estruturasdados.pdf>. Acesso em 25/03/2021.